to C; but A didn't finish the batch, and when it restarts, it does so from the first element of the 10. After the restart, B gets the 10 elements, and so does C. If that is the case, you have at-least-once semantics. However, if B keeps track of what it has received and sent to C, it can verify that the first 5 elements have been sent to C. If that is the case, you have effectively-once semantics.

Akka Transactional has effectively-once semantics, and its API is very similar to what you learned in the previous section. You use the following:

- `Transactional.source` instead of `Consumer.commitableSource`
- `TransactionalMessage` instead of `CommittableMessage`
- `Transactional.sink` instead of `Producer.commitableSink`

You can use these just like the committable versions, with one exception: the transactional ID. This ID must be unique at runtime, and two applications *cannot* use it simultaneously. Attempting to do so throws a fatal `ProducerFencedException` when Kafka detects a running process using that transactional ID. So, if you are using multiple instances of the same application, you must assign a unique ID to each.

Listing 14.4 Transactional source and sink

```
object ConsumerProducerEffectivelyOnce {

 val transactionalId = args(0)
```